



PRÉ-RENTREE 2026-2027

SÉANCE 4 - LISTES, ENREGISTREMENTS ET DONNÉES CSV

Dossier personnalisé - Ahmad BELDI - durée : 2 heures

Parcours : Fondations guidées

Objectif personnel de la séance 4 : stabiliser définitivement l'**indexation** et **len**, puis utiliser une liste de dictionnaires pour charger et analyser le fichier `mesures_capteurs.csv`.

Production à conserver : le fichier `S4_structures_csv_Ahmad.py`, avec cinq accès corrects, un chargement CSV typé, un filtre d'alertes, une moyenne et des tests simples.

Le fil des quatre séances

S1	suivre l'état des variables et choisir une condition ;	aujourd'hui, chaque ligne CSV deviendra un état structuré ;
S2	parcourir une liste avec une boucle ;	aujourd'hui, la boucle parcourra des enregistrements ;
S3	écrire des fonctions qui renvoient une valeur ;	aujourd'hui, chaque traitement sera placé dans une fonction ;
S4	accéder, copier, enregistrer, importer et filtrer.	preuve finale : un analyseur de table exécutable.

1. Feuille de route des 120 minutes

Temps	Travail	Preuve attendue
0-10 min	Rituel : indices, longueur et dernier élément.	Cinq accès exacts.
10-28 min	Chaînes, tuples, listes et parcours.	Tableau comparatif complété.
28-45 min	Mutation, alias et copie.	Liste d'origine préservée.
45-65 min	Dictionnaire : construire une mesure.	Enregistrement de cinq clés.
65-70 min	Pause.	—
70-88 min	Lire le CSV et convertir les types.	Liste de dictionnaires typés.
88-100 min	Filtrer et calculer une moyenne.	Nouvelle liste + résultat contrôlé.
100-112 min	Trier, détecter un doublon, contrôler.	Trois contrôles exécutés.
112-120 min	Cartes de rappel et exit ticket.	Explication individuelle.

Critères de réussite : au moins cinq accès sans erreur ; distinction claire entre `len(L)` et le dernier indice ; aucune mutation involontaire ; valeurs numériques converties avant calcul ; filtre produisant une nouvelle liste ; une erreur de code expliquée précisément.

Consigne : travailler sans ordinateur pendant 7 minutes. Exécuter ensuite uniquement pour contrôler.

Pour $L = [4, 8, 15, 16]$, compléter sans exécuter.

Erreur à diagnostiquer

1. Quelle valeur pense-t-il demander ? _____

3. Écrire la correction : _____

Repère à mémoriser : une liste de longueur n possède les indices $0, 1, \dots, n-1$. La longueur compte les éléments ; elle n'est pas un indice valide.

Checkpoint personnel

Nombre d'accès corrects sur les cinq accès essentiels : _____ / 5.

Certitude : ☐1 ☐2 ☐3 ☐4

Contrôle : _____

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir `temperature` et `humidite` avant tout calcul.

3. Chaîne, tuple ou liste ? - 18 min

Ces trois structures sont **ordonnées** et s'indexent. La différence essentielle porte sur la **mutation** : une liste peut être modifiée ; une chaîne et un tuple ne le peuvent pas.

Structure	Exemple	Accès	Modification directe ?
Chaîne	"CAPTEUR"	mot[0]	
Tuple	(3, 5)	point[1]	
Liste	[28.4, 31.2, -1.5]	temperatures[-1]	

A. Accéder et décomposer

1. Pour `mot = "CAPTEUR"`, écrire `mot[0]`, `mot[3]` et `len(mot)`.
2. Pour `point = (3, 5)`, compléter : `x, y = point` donne `x = _____` et `y = _____`.
3. Pour `temperatures = [28.4, 31.2, -1.5]`, écrire le dernier élément de deux manières différentes.

B. Construire une nouvelle liste sans toucher à l'ancienne

On veut doubler chaque valeur de `valeurs`.

```
valeurs = [3, 7, 9]
doubles = []
for valeur in valeurs:
    _____
```

1. Compléter une seule instruction dans la boucle.
2. Écrire la valeur finale de `doubles` : _____
3. Vérifier que `valeurs` est restée inchangée : _____

Preuve attendue : je peux expliquer pourquoi le programme construit une nouvelle liste et ne remplace aucun élément de la liste source.

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir `temperature` et `humidite` avant tout calcul.

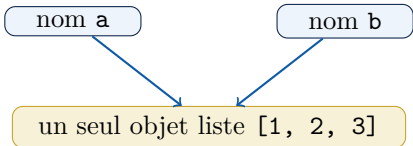
4. Mutation, alias et copie - 17 min

A. Deux noms pour une seule liste

Prévoir avant d'exécuter.

```
a = [1, 2]
b = a
b.append(3)
print("a =", a)
print("b =", b)
```

Question	Réponse avant exécution
Valeur affichée pour a	
Valeur affichée pour b	
Nombre réel d'objets liste	
Pourquoi a change-t-elle ?	



B. Créer une liste indépendante

Compléter la fonction du fichier S4_structures_csv_Ahmad.py.

```
def ajouter_sans_modifier(valeurs, nouvelle):
    copie = _____
    _____
    return _____
```

N°	Appel	Résultat attendu	Liste source après appel
1	ajouter_sans_modifier([1, 2], 3)		
2	ajouter_sans_modifier([], 5)		
3	ajouter_sans_modifier([-1], -2)		

Contrôle obligatoire : après l'appel, imprimer la liste source et la liste renvoyée. Si les deux ont été modifiées, la copie n'est pas correcte.

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir `temperature` et `humidite` avant tout calcul.

5. Un dictionnaire pour représenter une mesure - 20 min

Une liste repère une valeur par sa **position**. Un dictionnaire repère une valeur par une **clé**. Une ligne du fichier de capteurs sera représentée par un dictionnaire de cinq clés.

A. Construire l'enregistrement

Compléter avec les données : identifiant C01, date 2026-08-24, température 28.4, humidité 60, statut OK.

```
mesure = {  
    "id": _____,  
    "date": _____,  
    "temperature": _____,  
    "humidite": _____,  
    "statut": _____  
}
```

Clé	Valeur écrite	Type attendu	Contrôle avec <code>type</code>
<code>id</code>		<code>str</code>	
<code>date</code>		<code>str</code>	
<code>temperature</code>		<code>float</code>	
<code>humidite</code>		<code>int</code>	
<code>statut</code>		<code>str</code>	

B. Accéder par une clé

- Écrire l'expression qui lit la température : _____
- Écrire l'expression qui remplace le statut par "ALERTE" : _____
- Compléter le parcours :

```
for cle, valeur in mesure._____.():  
    print(cle, valeur)
```

C. Réutiliser les fonctions de S3

Compléter dans le fichier Python :

```
def creer_mesure(identifiant, date, temperature, humidite, statut):  
    # renvoyer un dictionnaire de cinq cles  
    _____
```

Preuve attendue : l'appel `creer_mesure(...)` renvoie un dictionnaire ; aucune valeur n'est seulement affichée à la place d'être renvoyée.

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir `temperature` et `humidite` avant tout calcul.

6. Lire une table CSV et convertir les types - 18 min

Fichier à ouvrir : mesures_capteurs.csv. Ne pas le renommer.

```
id,date,temperature,humidite,statut
C01,2026-08-24,28.4,60,OK
C02,2026-08-24,31.2,52,ALERTE
C03,2026-08-24,-1.5,80,ALERTE
```

A. Lire la structure avant de coder

- Combien de colonnes possède la table? _____
- Quelles colonnes doivent être converties avant un calcul? _____
- Pourquoi "28.4" + "1" ne calcule-t-il pas 29,4? _____

B. Compléter le chargeur

```
def charger_mesures(chemin):
    mesures = []
    with open(chemin, newline="", encoding="utf-8") as fichier:
        lecteur = csv._____(fichier)
        for ligne in lecteur:
            mesure = creer_mesure(
                ligne["id"],
                ligne["date"],
                _____(ligne["temperature"]),
                _____(ligne["humidite"]),
                ligne["statut"]
            )
            mesures._____(mesure)
    return mesures
```

C. Contrôles immédiatement après le chargement

Compléter avant d'exécuter.

Contrôle	Résultat attendu	Obtenu
type(mesures)		
type(mesures[0])		
type(mesures[0]["temperature"])		
type(mesures[0]["humidite"])		

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire len(L)-1 avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir temperature et humidite avant tout calcul.

7. Filtrer les alertes et calculer une moyenne - 12 min

A. Filtrer sans modifier la table source

```
def filtrer_alertes(mesures):
    alertes = []
    for mesure in mesures:
        if _____:
            _____
    return alertes
```

1. Écrire la condition à tester.
2. Ajouter l'enregistrement à la nouvelle liste.
3. Avant d'exécuter, relever dans le CSV les identifiants attendus : _____

B. Réutiliser l'accumulateur de S2

```
def moyenne_temperature(mesures):
    if len(mesures) == 0:
        return None
    total = 0
    for mesure in mesures:
        total += _____
    return total / _____
```

N°	Test à préparer	Résultat attendu	Validé
1	moyenne_temperature([])		☐
2	Nombre de lignes chargées		☐
3	Nombre d'alertes		☐
4	Moyenne comprise entre mini- mum et maximum	oui / non	☐

Contrôle de vraisemblance : une moyenne des températures doit être comprise entre la plus petite et la plus grande température du fichier. Ce contrôle ne remplace pas le calcul, mais il détecte certains résultats impossibles.

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l'opération modifie la liste ; D - écrire les cinq clés de l'enregistrement ; E - convertir **temperature** et **humidite** avant tout calcul.

8. Trier, repérer un doublon et contrôler - 12 min

A. Trier sans apprendre encore un algorithme de tri

La séance 5 étudiera les algorithmes. Aujourd’hui, on utilise l’outil Python `sorted` et une fonction-clé déjà fournie.

```
def cle_temperature(mesure):  
    return mesure["temperature"]  
  
mesures_triees = sorted(mesures, key=cle_temperature)
```

- 1. L’instruction modifie-t-elle `mesures` ou construit-elle une nouvelle liste ? _____
- 2. Après exécution, noter l’identifiant de la première mesure triée : _____
- 3. Noter l’identifiant de la dernière : _____

B. Détecter un identifiant répété

Compléter avec les identifiants : `["C01", "C02", "C01", "C03"]`.

```
identifiants = ["C01", "C02", "C01", "C03"]  
vus = []  
doublons = []  
for identifiant in identifiants:  
    if identifiant in vus:  
        _____  
    else:  
        _____
```

Identifiant dupliqué détecté : _____

C. Tableau de cohérence

Pour chaque anomalie, écrire le contrôle à réaliser.

Anomalie possible	Contrôle précis
Une clé <code>temperature</code> manque	
La valeur <code>"abc"</code> apparaît en température	
Le statut vaut <code>"INCONNU"</code>	
Le même identifiant apparaît deux fois	

Aides graduées - une seule à la fois : A - placer les indices sous les valeurs ; B - écrire `len(L)-1` avant le dernier accès ; C - demander si l’opération modifie la liste ; D - écrire les cinq clés de l’enregistrement ; E - convertir `temperature` et `humidite` avant tout calcul.

9. Atelier d'intégration : produire l'analyseur - 18 min

Fichier de travail : `S4_structures_csv_Ahmad.py`. Exécuter les étapes dans l'ordre ; ne pas passer à la suivante tant que la preuve demandée n'est pas obtenue.

Étape	Action exacte	Preuve à conserver
1	Terminer <code>ajouter_sans_modifier</code> .	source et résultat imprimés séparément ;
2	Terminer <code>creer_mesure</code> .	cinq clés et types affichés ;
3	Terminer <code>charger_mesures</code> .	première et dernière ligne lues ;
4	Terminer <code>filtrer_alertes</code> .	liste d'identifiants obtenue ;
5	Terminer <code>moyenne_temperature</code> .	cas vide + fichier réel ;
6	Trier avec <code>sorted</code> .	ordre source inchangé ;
7	Écrire au moins six tests.	six validations conservées.

Matrice de tests à remplir avant le code définitif

N°	Entrée / action	Résultat attendu	Validé
1	Copie + ajout sur [1,2]		☐
2	Accès au dernier élément		☐
3	Chargement du fichier CSV		☐
4	Type de la température		☐
5	Filtre des alertes		☐
6	Cas vide de la moyenne		☐

Journal de débogage

Symptôme	Hypothèse	Une modification	Résultat
----------	-----------	------------------	----------

Règle : un test qui échoue n'est pas effacé. Il sert à localiser le premier écart entre le résultat prévu et le résultat obtenu.

10. Cartes de rappel, exit ticket et bilan - 8 min

Carte A - Index

Premier indice : 0. Dernier indice : `len(L)-1`. Dernier élément : `L[-1]`.

Carte B - Alias

`b = a` ne copie pas une liste : deux noms désignent le même objet. Pour une copie simple : `a.copy()`.

Carte C - Dictionnaire

Une valeur se lit par sa clé : `mesure["temperature"]`. Parcours : `mesure.items()`.

Carte D - CSV

`DictReader` produit d'abord des chaînes. Convertir avec `float` et `int` avant les calculs.

Exit ticket - sans aide

1. Une liste contient n éléments. Quel est son dernier indice ? _____

2. Prédire `a` après `a=[1]` ; `b=a` ; `b.append(2)`. _____

3. Écrire l'accès à l'humidité d'un dictionnaire `mesure`. _____

4. Pourquoi faut-il convertir la température lue dans un CSV ?

5. Citer un contrôle que tu garderas pour la séance 5.

Ma preuve de progression

Je peux...	Pas encore	Avec aide	Seul	Expliquer
utiliser indices et <code>len</code>	☐	☐	☐	☐
distinguer alias et copie	☐	☐	☐	☐
construire un dictionnaire	☐	☐	☐	☐
charger et convertir un CSV	☐	☐	☐	☐
filtrer sans modifier la source	☐	☐	☐	☐

Fichier à conserver pour S5 : `S4_structures_csv_Ahmad.py` et `mesures_capteurs.csv`. La partie fiable de ce programme servira de base au mini-projet final.

Séance 4/5 - Ahmad BELDI | page 10/10